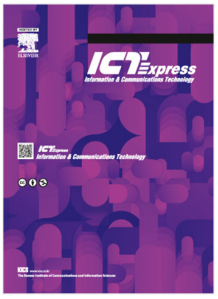
Journal Pre-proof



Utilization-aware multipath traffic splitting over delay-feasible paths for LEO satellite networks

Hyejin Kim, Hyunchul Baek, Seunghyun Yoon, Hyuk Lim

PII: S2405-9595(26)00153-0
DOI: https://doi.org/10.1016/j.icte.2026.08.003
Reference: ICTE 1094

To appear in: *ICT Express*

Received date : 10 May 2026
Revised date : 19 July 2026
Accepted date : 10 August 2026

Please cite this article as: H. Kim, H. Baek, S. Yoon et al., Utilization-aware multipath traffic splitting over delay-feasible paths for LEO satellite networks, *ICT Express* (2026), doi: https://doi.org/10.1016/j.icte.2026.08.003.

Utilization-Aware Multipath Traffic Splitting over Delay-Feasible Paths for LEO Satellite Networks

Hyejin Kim[a], Hyunchul Baek[b], Seunghyun Yoon[a], Hyuk Lim[a,*]

[a]*Institute for Energy AI, Korea Institute of Energy Technology (KENTECH), Naju, Republic of Korea*
[b]*Korea Aerospace Research Institute (KARI), Daejeon, Republic of Korea*

Abstract

Modern low Earth orbit (LEO) satellite networks use dense inter-satellite links (ISLs) and multipath connectivity to support low-latency communication. This paper proposes utilization-aware multipath traffic splitting over delay-feasible paths in LEO satellite networks. We construct delay-feasible candidate paths using a K-shortest path procedure and apply an L2 traffic splitting method that minimizes aggregate squared link utilization. Compared with Equal Split and LP Split, the proposed L2 splitting method better balances traffic across the network. Simulations on a 72-node LEO topology show that L2 splitting with $K = 3$ reduces average delay from 74.85 ms to 62.40 ms and improves delivery ratio from 66.5% to 82.8% compared with single-path routing. The robustness of the proposed method is further evaluated under realistic traffic scenarios, against adaptive routing baselines, under warm-started re-optimization, and at larger constellation scales. These results suggest that L2-based traffic splitting is a practical approach for software-defined traffic engineering in future LEO satellite networks.

Keywords: Satellite Networks, Link Capacity Dimensioning, Multipath Routing, SDN, QoS

1. Introduction

Satellite networks have evolved from simple bent-pipe relays into dynamic, interconnected systems spanning multiple orbital regimes. Low Earth orbit (LEO) satellite networks offer low latency and high throughput, and increasingly support applications such as global broadband connectivity, Earth observation, and real-time data relay, each requiring quality-of-service (QoS) guarantees on rate, delay bound, and jitter. Modern LEO constellations comprise thousands of satellites, and this scale is expected to keep growing, making efficient traffic engineering important. However, routing in LEO networks is inherently difficult due to rapidly changing topology, long propagation distances, and physical constraints. To address this, software-defined networking (SDN) has been explored as a promising solution. By decoupling the data and control planes, SDN enables a centralized controller to maintain a global network view and perform real-time traffic engineering.

The main contribution of this paper is a practical traffic engineering framework for centralized SDN-based LEO satellite networks that combines delay-feasible candidate-path construction with capacity-aware multipath traffic splitting. The framework is extensively evaluated under realistic traffic scenarios, at larger constellation scales, against adaptive routing baselines, including warm-started re-optimization.

2. Related Work

2.1. QoS Routing in Satellite Networks

QoS-aware routing approaches aim to minimize congestion and meet application-specific QoS goals. QOGMP [1] used deep reinforcement learning to compute link weights for QoS-aware multi-path traffic scheduling in SDN. fybrrLink [2] combined Bresenham's and Dijkstra's algorithms for fast QoS-aware routing in SDN-enabled LEO satellite networks. POMAP [3] formulated packet routing as a Pareto-optimal multi-agent reinforcement learning problem over delay, energy consumption, and packet loss.

*Corresponding author
Email addresses: hyejinkim@kentech.ac.kr (Hyejin Kim), sn100@kari.re.kr (Hyunchul Baek), syoon@kentech.ac.kr (Seunghyun Yoon), hlim@kentech.ac.kr (Hyuk Lim)

2.2. Multipath Routing in Satellite Networks

Multipath routing has been studied to enhance throughput, reduce latency, and improve robustness in satellite networks. This work [4] proposed a two-stage multipath routing framework that combines a Pareto-optimal MRO algorithm with a Stackelberg game framework for path selection and traffic scheduling. DBPR [5] employed distance-based link weights to distribute traffic over multiple back-pressure paths within a bounded source-destination region. DOBP [6] augmented back-pressure routing by incorporating queue backlog, residual propagation delay, and a minimum load factor into its link-weight computation.

2.3. SDN-Based Routing for Satellite Networks

SDN-based architectures have been proposed to support flexible and centralized control in satellite networks. An SDN-based architecture for integrated satellite–terrestrial networks [7] computed multi-objective, fragmentation-aware routes for elastic data flows. Dyna-STN [8] introduced a virtual overlay network of fixed virtual nodes to shield the dynamic satellite topology. Subsequent work [9] further demonstrated Dyna-STN's routing and packet-forwarding performance on realistic, large-scale satellite–terrestrial network topologies.

3. System Model

3.1. Network and Link Model

We model the satellite network as a directed graph $G = (V, E)$, where V and E denote the sets of satellites and ISLs, and each link $e \in E$ has capacity C_e determined by its distance d_e. For RF ISLs, the capacity is modeled as

$$C_e = C_{\text{ref}} \left(\frac{d_{\text{ref}}}{d_e} \right)^n \eta, \tag{1}$$

where C_{ref} is the reference capacity at distance d_{ref}, n is the path loss exponent, and η is the spectral efficiency.

3.2. Traffic and Delay Model

Let $\mathcal{D}$ denote the set of traffic demands, each represented by (s_d, t_d, f_d, T_d), where s_d and t_d are the source and destination nodes, f_d is the requested traffic volume, and T_d is the end-to-end delay bound.

Each ISL is modeled as an M/M/1/B finite-buffer queue, where B denotes the finite buffer size. Given packet size L, the service rate and traffic intensity of link e are

$$\mu_e = \frac{C_e}{L}, \qquad \rho_e = \frac{\lambda_e}{\mu_e}, \tag{2}$$

where λ_e is the aggregate packet arrival rate on link e. The blocking probability is

$$p_{B,e} = \begin{cases} \dfrac{1}{B+1}, & \rho_e = 1, \\ \dfrac{\rho_e^B(1-\rho_e)}{1-\rho_e^{B+1}}, & \rho_e \neq 1. \end{cases} \tag{3}$$

The mean queuing delay excluding service time is

$$W_{q,e} = \frac{L_{q,e}}{\lambda_e(1-p_{B,e})} - \frac{1}{\mu_e}, \tag{4}$$

where $L_{q,e}$ is the mean queue length.

For a path p, the end-to-end delay is

$$T_p = \sum_{e \in p} \left(\frac{d_e}{c} + W_{q,e} + \frac{L}{C_e} \right), \tag{5}$$

where c is the speed of light. For path $p_{d,k}$, the packet survival probability is

$$q_{d,k} = \prod_{e \in p_{d,k}} (1 - p_{B,e}). \tag{6}$$

Given a path-flow allocation $\{x_{d,k}\}$, the achieved traffic of demand d is

$$f_d^{\text{rx}} = \sum_{k=1}^{K_d} x_{d,k} q_{d,k}. \tag{7}$$

4. Proposed Optimization Framework

4.1. Delay-Feasible Candidate Path Construction

To construct a feasible set of candidate paths for each demand, we first generate delay-feasible loopless paths over the satellite network. Let $G = (V, E)$ denote the directed network graph, where each link $e \in E$ is associated with a link delay. For each traffic demand $d \in \mathcal{D}$, let $d = (s_d, t_d, f_d, T_d)$ denote its source node, destination node, traffic volume, and end-to-end delay bound.

For demand d, we denote the candidate path set by

$$\mathcal{P}_d = \{p_{d,1}, \ldots, p_{d,K_d}\}, \qquad K_d \leq K, \tag{8}$$

where K is the target maximum number of paths. The inequality $K_d \leq K$ accounts for the case where fewer than K delay-feasible paths exist under the bound T_d.

Algorithm 1 describes the procedure for generating delay-feasible candidate paths. Here, Yen's algorithm [10] has a worst-case complexity of $O(K|V|(|E| + |V| \log |V|))$ per demand. Since queuing delay is load-dependent and cannot be determined during candidate-path generation, the factor α with $0 < \alpha \leq 1$ reserves delay margin for subsequent queuing delay. Unless otherwise stated, the experiments use $\alpha = 1$, and a sensitivity analysis for α is provided in Section 5.3.

Algorithm 1: Delay-Feasible Candidate Paths

Input: Graph $G = (V, E)$ with link delays; demand $d = (s_d, t_d, f_d, T_d)$; delay margin α; maximum number of paths K

Output: Delay-feasible path set $\mathcal{P}_d$

$\{p_1, \ldots, p_K\} \leftarrow \mathrm{Yen}(G, s_d, t_d, K)$; `// K shortest loopless paths ranked by propagation-and-transmission delay`

$\mathcal{P}_d \leftarrow \{\, p_i : \mathrm{delay}(p_i) \le \alpha T_d,\ i = 1, \ldots, K \,\}$;

return $\mathcal{P}_d$;

4.2. Capacity-Aware Traffic Splitting Formulation

Given the precomputed candidate path set $\mathcal{P}_d$ for each demand d, we formulate a capacity-aware traffic splitting problem for link capacity dimensioning [11]. Let $x_{d,k} \ge 0$ denote the amount of traffic assigned to path $p_{d,k} \in \mathcal{P}_d$. The traffic assigned to the candidate paths of each demand must satisfy the flow conservation constraint

$$\sum_{k=1}^{K_d} x_{d,k} = f_d, \qquad \forall d \in \mathcal{D}. \tag{9}$$

To characterize link usage, we define the path–link incidence indicator $a_{e,d,k} = \mathbf{1}[e \in p_{d,k}]$. The total load imposed on link e is

$$y_e = \sum_{d \in \mathcal{D}} \sum_{k=1}^{K_d} a_{e,d,k} x_{d,k}, \tag{10}$$

and the normalized link utilization is

$$\rho_e \triangleq \frac{y_e}{C_e} = \frac{1}{C_e} \sum_{d \in \mathcal{D}} \sum_{k=1}^{K_d} a_{e,d,k} x_{d,k}, \tag{11}$$

where C_e is the physical capacity of link e.

The L2-based traffic splitting problem is formulated as the following convex quadratic program (QP):

$$\begin{aligned} \min_{\mathbf{x}} \quad & \sum_{e \in \mathcal{E}} \rho_e^2 + \varepsilon \|\mathbf{x}\|_2^2 \\ \text{s.t.} \quad & \sum_{k=1}^{K_d} x_{d,k} = f_d, \qquad \forall d \in \mathcal{D}, \\ & x_{d,k} \ge 0, \qquad \forall d \in \mathcal{D},\ k = 1, \ldots, K_d. \end{aligned} \tag{12}$$

Here, $\varepsilon \ge 0$ is a Tikhonov regularization coefficient used to improve numerical stability and to avoid excessively unbalanced path-level allocations. In all experiments, ε is set to 10^{-8}. Because ρ_e is a linear function of $\mathbf{x}$, the objective in (12) is convex quadratic and the constraints are linear.

For compact matrix notation, stack all path-flow variables into $\mathbf{x}$. Let $\mathbf{A}_{\mathrm{link}}$ be the link–path incidence matrix, $\mathbf{y} = \mathbf{A}_{\mathrm{link}}\mathbf{x}$, and $\mathbf{W} = \mathrm{diag}(1/C_e)\mathbf{A}_{\mathrm{link}}$, so that the objective is $F(\mathbf{x}) = \|\mathbf{W}\mathbf{x}\|_2^2 + \varepsilon\|\mathbf{x}\|_2^2$.

4.3. Projected-Gradient Solver

The problem in (12) can be solved using a standard centralized QP solver (CVXPY with the CLARABEL backend), or via an iterative projected gradient descent (PGD) solver that updates path-flow variables using the L2 objective's gradient and projects each demand's allocation onto the feasible simplex, providing a simple implementation that supports warm-starting and admits a distributed link-utilization-feedback interpretation.

The gradient of the objective with respect to $x_{d,k}$ is

$$g_{d,k} = \frac{\partial F}{\partial x_{d,k}} = 2 \sum_{e \in p_{d,k}} \frac{y_e}{C_e^2} + 2\varepsilon x_{d,k}. \tag{13}$$

For each demand d, define the feasible simplex

$$\mathcal{X}_d = \left\{ \mathbf{x}_d \in \mathbb{R}^{K_d} :\ x_{d,k} \ge 0,\ \sum_{k=1}^{K_d} x_{d,k} = f_d \right\}. \tag{14}$$

At iteration r, the PGD update is

$$\tilde{x}_{d,k}^{(r+1)} = x_{d,k}^{(r)} - \eta\, g_{d,k}^{(r)}, \tag{15}$$

$$\mathbf{x}_d^{(r+1)} = \Pi_{\mathcal{X}_d}\left(\tilde{\mathbf{x}}_d^{(r+1)}\right), \tag{16}$$

where $\Pi_{\mathcal{X}_d}(\cdot)$ denotes Euclidean projection onto the simplex $\mathcal{X}_d$, which simultaneously enforces non-negativity and flow conservation without separate clipping or equality-dual updates.

A safe fixed step size can be selected using the Lipschitz constant of the gradient,

$$L = 2\sigma_{\max}^2(\mathbf{W}) + 2\varepsilon, \tag{17}$$

where $\sigma_{\max}(\mathbf{W})$ is the largest singular value of $\mathbf{W}$. In the experiments, $\sigma_{\max}(\mathbf{W})$ is estimated with 15 power-iteration steps, and the primal step size is set to $\eta = 0.9/L$. The solver is initialized with the equal split $x_{d,k}^{(0)} = f_d/K_d$ and terminates when the relative change of the objective between consecutive iterations falls below 10^{-7} or after a maximum of 3,000 iterations.

5. Performance Evaluation

5.1. Experimental Setup and Baselines

The satellite network is modeled as a Walker Delta constellation with 72 satellites across 8 orbital planes at an altitude of 1,200 km and an inclination of 55°,

Table 1: Routing performance across K and splitting methods.

K	Split	Delay (ms)	Rx (Mbps)	Deliv. (%)	Sat.
–	Single-path	74.85	133.0	66.5	28/36
1	Equal (=L2=LP)	74.85	145.1	72.6	28/36
2	Equal	76.22	144.0	72.0	27/36
2	LP	75.16	148.5	74.2	28/36
2	L2 (proposed)	**67.74**	**152.9**	**76.4**	**30/36**
3	Equal	75.24	150.6	75.3	27/36
3	LP	76.03	154.9	77.4	27/36
3	**L2** (proposed)	**62.40**	**165.6**	**82.8**	**31/36**
4	Equal	77.01	154.8	77.4	28/36
4	LP	78.22	156.9	78.4	28/36
4	**L2** (proposed)	**62.31**	**166.5**	**83.3**	**31/36**

with a maximum ISL distance of 4,000 km. The RF ISL capacity follows a distance-dependent model with a reference distance of 2,500 km, a reference capacity of 1.0 Gbps, a path loss exponent of 2, and a spectral efficiency of 0.8. Traffic demands are generated between source–destination satellite pairs. Unless otherwise stated, we use 36 demands of 200 Mbps each, with a delay bound of 100 ms and a packet size of 1,500 bytes. The buffer size of each ISL queue is set to 800 packets. Routing performance is evaluated using average end-to-end delay, received traffic, delivery ratio, and the number of demands satisfying the delay bound. We compare the proposed method with two traffic splitting strategies: Equal Split, which uniformly distributes each demand across its candidate paths without optimization, and LP Split, which minimizes the maximum normalized link utilization. All methods use the same set of delay-feasible candidate paths.

5.2. End-to-End Routing Performance

Table 1 compares the end-to-end performance of single-path routing and multipath routing using Equal, LP, and L2 splitting. As K increases, L2 Split consistently achieves lower delay, higher received traffic, and higher delivery ratio than Equal and LP. Increasing K from 3 to 4 provides only marginal improvement, so $K = 3$ offers a practical balance between routing performance and computational complexity. The performance gain of L2 Split stems from minimizing the sum of squared link utilizations, which reduces traffic concentration and queuing delay.

5.3. Sensitivity to the Delay Margin α

Table 2 reports the sensitivity of the proposed method to the delay margin α. With $\alpha = 0.5$, longer paths are pruned, reducing the average number of candidate paths to 1.97 per demand, with average delay increasing only modestly to 65.29 ms. These results suggest

Table 2: Effect of α on routing performance.

α	Avg cand. paths	Demands $< K$ paths	Delay (ms)	Rx (Mbps)	Sat.
0.5	1.97	19	65.29	162.4	29/36
0.6	2.39	12	63.95	162.8	30/36
0.7	2.72	5	64.04	162.7	30/36
0.8	2.89	2	62.28	165.4	31/36
0.9	3.00	0	62.40	165.6	31/36
1.0	3.00	0	62.40	165.6	31/36

Table 3: Routing performance under realistic traffic.

	Heterogeneous			Hotspot			
Split	ms	%	Sat.	ms	%	Sat.	$\rho_e>1$
Single	77.85	63.0	26/36	80.62	68.1	28/36	16
Equal	74.81	69.1	27/36	76.30	68.6	30/36	12
LP	75.15	71.0	27/36	76.38	69.4	30/36	12
L2	**65.97**	**76.2**	**28/36**	**66.48**	**76.9**	**31/36**	**7**

that a smaller α reduces path diversity with little impact on performance, indicating that the proposed method is robust to the choice of α while providing additional protection against queuing-delay violations.

5.4. Performance under Realistic Traffic Scenarios

We evaluate the L2 splitting method under realistic traffic scenarios as summarized in Table 3. In the heterogeneous scenario, per-demand traffic is drawn uniformly from $\{50, 100, \ldots, 400\}$ Mbps, and per-demand delay bounds are drawn from $\{50, 100, 150\}$ ms, representing mixed QoS classes. Candidate paths are filtered by each demand's delay bound, with a fallback to the shortest path if no candidate satisfies it. L2 Split achieves the lowest average delay of 65.97 ms and the highest delivery ratio of 76.2%. In the hotspot scenario, half of the 36 demands are redirected so that their destinations are distributed across the 9 satellites of a single orbital plane. This concentrates traffic on the links serving that plane rather than spreading it network-wide. Single, Equal, and LP overload 16, 12, and 12 links. L2 reduces this to 7 overloaded links and improves delivery by more than 7 percentage points, demonstrating its effectiveness under concentrated traffic.

5.5. Warm-Start Evaluation

Since LEO network states typically change gradually over time, the previous solution can be used to warm-start the next optimization step. Table 4 reports the speedup achieved by warm-starting across five scenarios with increasing topology and traffic disruption. The resulting speedup ranges from 3.55× under capacity perturbation to 0.99× under combined stress. Specifically, warm-starting is most effective when the previous

Table 4: Warm-start convergence under network dynamics.

Scenario	Cold (iters)	Warm (iters)	Speedup
Capacity perturbation (±3%)	439	128	3.55×
Traffic fluctuation (±20%)	296	179	1.65×
Link failures (5 ISLs removed, 3 seeds)	486	451	1.09×
Snapshot sequence (6 transitions, 10 min)	303	296	1.03×
Combined stress (topology+failures+traffic)	276	279	0.99×

Table 5: Delay (ms) and delivery ratio (%) under dynamic scenarios.

Method	Snapshot seq.	Link failure	Traffic fluct.	Combined stress
BP-inspired [5]	55.47 (68.6)	75.66 (75.4)	67.19 (80.4)	62.11 (80.4)
DOBP-inspired [6]	55.53 (68.8)	75.33 (75.8)	67.14 (80.2)	61.81 (81.6)
L2 (proposed)	**49.96** (70.5)	**67.89** (78.4)	**63.75** (82.3)	**57.54** (82.4)

solution remains a good initial point for the new problem. Accordingly, the benefit is preserved when the set of candidate paths remains unchanged, as in capacity and traffic perturbations, but diminishes when the path set changes substantially, as in link failures, the snapshot sequence, and combined stress. Even in these worst cases, warm-start converges to a solution within 0.1% of the cold-start solution. A cold-start optimization takes about 125 ms on average for this topology, indicating that re-optimization is limited by link-state collection latency rather than computation.

5.6. Comparison with Adaptive Routing Methods

Adaptive routing methods route reactively based on locally observed congestion. *BP-inspired* and *DOBP-inspired* are flow-level analogues of DBPR [5] and DOBP [6]. Each round, candidate paths are scored, and a small fraction of a demand's flow is shifted from the worst- to the best-scoring path. BP-inspired scores paths by multiplying bottleneck-link utilization by base propagation delay normalized by the demand's mean candidate-path delay, whereas DOBP-inspired adds the two terms, with delay normalized by the demand's delay bound. Both are implemented as static, per-snapshot, path-level procedures. Table 5 reports the average delay across the four dynamic scenarios. For the BP-/DOBP-inspired methods, the best result among step sizes of {5%, 10%, 20%} is reported for each scenario. L2 achieves the lowest delay and the highest delivery ratio across all four scenarios because it jointly optimizes all traffic demands using centralized global link-state information, whereas the BP- and DOBP-inspired methods adjust each demand independently by shifting traffic among candidate paths.

5.7. Scalability to Larger Constellations

Table 6: Control-plane cost across constellation scales.

N	$\|E\|$	D	Path-gen (s)	Incidence (ms)	CVXPY (s)	PGD-dense (s)	PGD-sparse (s)
70	140	35	0.02	0.5	0.018	1.08	1.13
144	806	72	0.11	1.6	0.027	1.65	1.66
324	3,690	162	1.04	5.9	0.136	5.47	2.75
576	12,480	288	5.18	15.7	0.434	11.32	7.93
1,584	92,688	400	76.47	63.1	3.151	61.23	16.71

Table 7: Routing performance across constellation scales.

Method	N=70	144	324	576	1,584
Equal	115.11 (55.8)	56.35 (93.2)	46.38 (95.6)	46.29 (94.1)	47.33 (94.1)
LP	116.24 (55.8)	53.91 (94.3)	45.10 (97.4)	45.21 (94.7)	48.03 (94.1)
BP-inspired [5]	115.08 (59.1)	55.16 (95.0)	46.14 (96.9)	44.57 (96.0)	46.44 (94.8)
DOBP-inspired [6]	114.23 (59.2)	55.16 (95.0)	46.14 (96.9)	44.56 (96.0)	46.44 (94.8)
L2 (proposed)	**100.74** (**60.0**)	**50.45** (**95.1**)	**44.06** (**97.9**)	**43.54** (**96.7**)	**46.17** (**94.9**)

We regenerate Walker Delta constellations of increasing size up to 1,584 satellites. The number of demands scales proportionally at one demand per two satellites and is capped at 400 for the largest constellation to maintain computational tractability. The numbers of orbital planes and satellites per plane are increased together from 8×9 to 36×44. Table 6 reports the control-plane cost. Candidate-path generation dominates the runtime, increasing to 76.5 s, or 191 ms per demand, for the 1,584-satellite constellation, whereas incidence-matrix construction remains negligible, ranging from 0.5 to 63.1 ms. CLARABEL is the fastest optimizer at all tested scales, while sparse PGD reduces its optimization time from 61.2 s to 16.7 s for the largest constellation. Table 7 compares routing quality on the same topologies. L2 achieves the lowest delay and the highest delivery ratio at every tested scale, confirming that its routing advantage extends beyond the 72-satellite configuration.

6. Discussion

Loss and Failure Recovery Behavior. Since L2 splitting performs actual traffic division, buffer-overflow losses directly reduce delivered traffic (Eq. (7)); recovery is left to end-to-end transport, occurring only at the next re-optimization cycle. **Topology Dynamics and Re-Optimization Period.** The framework operates in a time-slotted control loop that periodically refreshes link-state information, recomputes candidate paths, and re-optimizes traffic allocation using a warm start. Since topology evolution can substantially change the candidate-path set within approximately 10 minutes (Section 5.5), the re-optimization period should remain well below this topology coherence time. Because satellite orbital motion is deterministic and known

in advance, the network topology at any future re-optimization instant can be predicted, allowing the corresponding candidate-path set to be pre-computed ahead of time rather than generated reactively. We leave this predictive pre-computation strategy as future work.

Physical-Layer Model. The distance-dependent capacity model (Eq. (1)) is a first-order abstraction ignoring antenna gain, pointing loss, interference, and adaptive modulation/coding. Since the optimization consumes only C_e per link, any refined capacity model can be substituted without changing the formulation.

Control-Plane Overhead. A complete online cycle comprises link-state collection, candidate-path generation, incidence-matrix construction, optimization, and flow-rule dissemination, dominated by signaling latency even at the largest tested scale (Table 6). For constellations beyond a single controller's scope, the PGD update admits a decentralized interpretation: each link advertises a congestion price and each source updates its own flows.

7. Conclusion

This study proposed utilization-aware multipath traffic splitting over delay-feasible paths in LEO satellite networks using K-shortest candidate paths and an L2 objective over normalized link utilization. The experiments show that L2 splitting consistently outperforms Equal, LP, and adaptive BP-/DOBP-inspired baselines across diverse traffic scenarios and constellation scales. A centralized QP solver (CVXPY/CLARABEL) remains the fastest optimization method at all tested scales, while the sparse PGD solver benefits from warm-starting and provides a decentralized alternative for larger constellations. These results demonstrate that L2-based traffic splitting is a practical and scalable approach for online SDN-based traffic engineering in future LEO satellite networks.

Acknowledgments

This work was partially supported by KRIT grant funded by the Korea government (DAPA) (KRIT-CT-22-040).

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author(s) used ChatGPT by OpenAI for language polishing and editorial assistance. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] Y. Guo, G. Hu, D. Shao, QOGMP: QoS-oriented global multipath traffic scheduling algorithm in software defined network, Sci. Rep. 12 (1) (2022) 14824.
[2] P. Kumar et al., fybrrLink: Efficient QoS-aware routing in SDN enabled future satellite networks, IEEE Trans. Netw. Serv. Manag. 19 (3) (2022) 2107–2118.
[3] S. Li, G. Wu, Q. Wu, R. Wang, H. Zhang, Efficient packet routing for large-scale LEO satellite networks: A Pareto-optimal MARL approach with queueing theory, IEEE Internet Things J. 12 (22) (2025) 46675–46691.
[4] S. Li, Q. Wu, R. Wang, L. Chen, H. Zhang, Efficient multipath differential routing and traffic scheduling in ultra-dense LEO satellite networks: A DRL with Stackelberg game approach, IEEE Trans. Mobile Comput. 24 (11) (2025) 12424–12440.
[5] X. Deng, L. Chang, S. Zeng, L. Cai, J. Pan, Distance-based back-pressure routing for load-balancing LEO satellite networks, IEEE Trans. Veh. Technol. 72 (1) (2023) 1240–1253.
[6] W. Deng, X. Lv, J. Liu, Delay-optimized backpressure algorithm for load balancing in LEO satellite networks, IEEE Commun. Lett. 30 (2026) 922–926.
[7] Q. Guo et al., SDN-based end-to-end fragment-aware routing for elastic data flows in LEO satellite-terrestrial network, IEEE Access 7 (2019) 396–410.
[8] S. Li, Q. Wu, R. Wang, Dynamic discrete topology design and routing for satellite-terrestrial integrated networks, IEEE/ACM Trans. Netw. 32 (5) (2024) 3840–3853.
[9] S. Li, Q. Wu, R. Wang, R. Lu, Toward networking and routing in 6G satellite-terrestrial integrated networks: Current issues and a potential solution, IEEE Commun. Mag. 63 (2025) 92–98.
[10] J. Y. Yen, Finding the K shortest loopless paths in a network, Manage. Sci. 17 (11) (1971) 712–716.
[11] M. Pióro, D. Medhi, Routing, Flow, and Capacity Design in Communication and Computer Networks, Morgan Kaufmann, 2004.

CRediT authorship contribution statement

Hyejin Kim: Conceptualization, Methodology, Software, Formal analysis, Investigation, Validation, Visualization, Writing – original draft.
Hyunchul Baek: Conceptualization, Resources, Validation, Writing – review & editing.
Seunghyun Yoon: Software, Investigation, Validation, Visualization, Writing – review & editing.
Hyuk Lim: Conceptualization, Methodology, Supervision, Project administration, Funding acquisition, Writing – review & editing.

[Declaration of competing interest]

The authors declare that there is no conflict of interest in this paper.